

Relatório Técnico

Arthur & Wesley

June 8, 2026

1 Representação e Estrutura da Trie

A eficiência da *Trie* reside na sua capacidade de compartilhar prefixos comuns entre diferentes palavras, reduzindo drasticamente o número de comparações de strings.

1.1 Escolha do Tamanho do Alfabeto

Para acomodar títulos de jogos que frequentemente mesclam letras e números (como "*Fifa 23*" ou "*Grand Theft Auto 5*"), o tamanho do alfabeto foi fixado em 36. Essa escolha engloba perfeitamente as 26 letras do alfabeto (case-insensitive) e os 10 dígitos numéricos. Símbolos especiais, pontuações e espaços são descartados para manter a estrutura compacta e focada na relevância textual.

1.2 Mapeamento de Caracteres no Array de Filhos

Cada `TrieNode` gerencia um array fixo de ponteiros de tamanho 36. O mapeamento de um caractere para o seu respectivo índice no array ocorre por meio de aritmética de ponteiros na função `getCharIndex`:

- **Letras ('a' a 'z'):** São mapeadas para o intervalo $[0, 25]$ através da operação $c - 'a'$.
- **Dígitos ('0' a '9'):** São mapeados para o intervalo $[26, 35]$ através da operação $(c - '0') + 26$.

Caracteres fora desses intervalos retornam -1 e são sumariamente ignorados durante os fluxos de inserção e busca, garantindo consistência operacional.

1.3 Conversão para Chaves de Busca

O método `toSearchKey` transforma qualquer string original de título ou prefixo seguindo três regras estritas: 1. Todas as letras são convertidas para minúsculas (`std::tolower`). 2. Caracteres de espaço são ignorados. 3. Apenas caracteres que satisfazem `std::isalpha` ou `std::isdigit` são concatenados na chave resultante.

2 Funcionamento Métodos

A *Trie* opera por meio de caminhamentos direcionados pelos caracteres da chave de busca gerada.

2.1 Método insert

O método recebe um ponteiro para um objeto `Game` existente na memória. A partir da chave gerada pelo título do jogo, o algoritmo desce a partir da raiz da árvore caractere por caractere. Se o ponteiro para o filho no índice correspondente for `nullptr`, um novo `TrieNode` é alocado dinamicamente. Ao alcançar o caractere final da chave, o nó correspondente é marcado com `isEndOfTitle = true` e o seu ponteiro interno `game` passa a apontar diretamente para o objeto do jogo original.

2.2 Método contains

O algoritmo realiza um caminhamento iterativo pelos nós da árvore guiado pelos índices dos caracteres. Se em qualquer ponto do caminho o ponteiro para o próximo nó for nulo, a função retorna imediatamente **false**. Caso o caminho seja percorrido integralmente, o método valida se o nó final representa o término de um título legítimo.

2.3 Método autocomplete

O processo de preenchimento automático opera em três etapas distintas: 1. O algoritmo consome o prefixo caractere por caractere para localizar o nó raiz da subárvore correspondente. Se o prefixo não existir na árvore, retorna-se um vetor vazio. 2. A partir do nó do prefixo, uma busca em profundidade (DFS) é disparada de forma recursiva para coletar todos os ponteiros de **Game** contidos na subárvore, acumulando-os em um vetor de resultados. 3. O vetor contendo todos os jogos encontrados é ordenado via *QuickSort* sob os critérios de ordenação decrescente de popularidade e, em caso de empate, ordem alfabética crescente das chaves de busca. Por fim, o método extrai e retorna um subvetor limitado aos primeiros k elementos solicitados.

3 Análise de Custo Computacional

Table 1: Complexidade das Operações na Trie

Operação	(Pior Caso)	(Médio Caso)
insert	$O(L)$	$O(L)$
contains	$O(L)$	$O(L)$
autocomplete	$O(p + s + r^2)$	$O(p + s + r \log r)$

3.1 Origem dos Custos

- **insert e contains:** O custo $O(L)$ decorre diretamente do laço que itera sobre os L caracteres da chave de busca.
- **autocomplete:** A primeira etapa de localização consome $O(p)$ passos. A exploração recursiva via DFS visita exatamente s nós, executando um laço interno de tamanho fixo 36 em cada um, resultando em um custo de $O(36s) = O(s)$. A etapa de ordenação dos r elementos encontrados via *QuickSort* possui custo médio de $O(r \log r)$ e pior caso de $O(r^2)$.

4 Comparação de Desempenho e Uso de Memória

4.1 Trie versus Solução Ingênua

Uma solução ingênua armazenaria os ponteiros em um vetor dinâmico (`std::vector<Game*>`). Para um banco de dados com N jogos:

- **contains:** A solução ingênua exige uma busca linear que inspeciona até N elementos, realizando comparações de strings de tamanho L em cada um, resultando em um custo de $O(N \cdot L)$. A *Trie* reduz esse custo para $O(L)$.
- **autocomplete:** A abordagem ingênua deve varrer todos os N jogos para verificar quais strings começam com o prefixo dado, demandando tempo $O(N \cdot p)$. Na *Trie*, a busca restringe-se localmente à subárvore do prefixo ($O(p + s)$), de modo que o algoritmo sequer toma conhecimento da existência dos outros $N - r$ jogos contidos no banco de dados.

4.2 Análise do Custo de Memória

O ganho substancial de velocidade da *Trie* cobra um preço em consumo de memória espacial. Enquanto a solução ingênua consome estritamente $O(N)$ espaço para armazenar os ponteiros no array, a *Trie* introduz uma sobrecarga estrutural por nó.

Cada `TrieNode` aloca:

$$(36 \times 8) + 8 + 1 = 297 \text{ bytes} \quad (1)$$

O fator redutor desse custo é o **compartilhamento de prefixos**, mitigando o crescimento puramente linear do número de nós em relação ao número de caracteres totais do banco de dados.

Contudo, o tamanho do alfabeto atua como um multiplicador fixo pesado. Mesmo que um nó possua apenas 1 filho ativo, ele é obrigado a carregar o custo de 36 ponteiros alocados na memória, gerando 35 ponteiros nulos ociosos. Assim, o esparsamento da árvore em níveis mais profundos realça a desvantagem espacial da *Trie* sobre o array linear compactado da solução ingênua.